

Table of Contents

Table of Contents.....	1
List of Figures.....	2
List Tables.....	3
I. Introduction.....	4
A. Background of the Study.....	4
B. Problem Statement.....	6
C. Objectives.....	6
D. Significance of the Project.....	7
II. Review of Related Literature.....	8
• Indoor Navigation Systems.....	8
• Dijkstra Algorithm.....	9
• Approaches to Design of Public Navigation Systems in Public Institutions.....	9
• Design and development of Google glass-based campus navigation system.....	10
• Indoor/Outdoor Positioning and Navigating System Set-Up in a University Campus.....	10
• How Navigation Systems can help with a user's Spatial Knowledge.....	11
• Detecting large crowds and rerouting to a faster path.....	12
III. Methodology.....	12
A. Conceptual Framework – IPO Chart/Diagram (Input-Process Output-Chart/Diagram)....	13
B. Hierarchy Chart.....	13
C. Algorithm.....	13
D. Flowchart.....	14
E. Pseudocode.....	19
IV. Results.....	31
V. Discussion of Results.....	34
VI. Analysis, Conclusion and Future Directives.....	35
References.....	36
Appendices.....	38
A. User's Manual.....	38
B. Source Code.....	38

List of Figures

IPO Chart	14
Hierarchy Chart	14
Flowchart Page 1	15
Flowchart Page 2	16
Flowchart Page 3	17
Flowchart Page 4	18
Flowchart Page 5	19
● Result Set 1	30
● Result Set 2	31
● Result Set 3	33

List Tables

Work breakdown

I. Introduction

A. Background of the Study

Technology has greatly advanced, particularly in the field of navigation. Evolving from maps to handheld devices capable of rendering a visual of an area right to handheld devices. This rapid advancement of mobile technology and the Internet of Things (IoT) has fundamentally changed how individuals interact with their environment. Globally, digital navigation has shifted from a luxury to a primary utility, with smartphone-based positioning systems becoming the standard for outdoor wayfinding (Zhang et al., 2017). Modern universities have adapted to the changing times by implementing navigation systems for their respective campuses. A notable precedent for this transition was set by a university in Norway through the deployment of MazeMap, a specialized indoor-outdoor positioning system designed specifically for campus navigation (Biczók et al., 2014). However, as modern universities are composed of dense campus environments, this makes it difficult for traditional navigation systems to work properly.

While Global Positioning Systems (GPS) are highly effective for open-space navigation, they struggle with proper network connectivity indoors. This transition from indoor to outdoor navigation remains a challenge for campus wayfinding. Wayfinding abilities differ for each individual and students, guests, and faculty alike have trouble navigating in unfamiliar environments while also dealing with high cognitive workload and "wayfinding anxiety" when attempting to locate specific classrooms in complex institutional layouts (Tahir & Krogstie, n.d.). Additionally, public institutions tend to have architectural designs that make navigation difficult, requiring specialized approaches in their navigation systems (Surina, 2019). Within De La Salle University (DLSU), students and visitors must rely on static directory boards or non-interactive maps, which fail to account for the high-density, multi-level buildings such as Andrew Gonzalez Hall or the Henry Sy Sr. Hall. There is a critical lack of "seamless" navigation that bridges

the transition from outdoor GPS tracking to precise indoor room-level guidance, a challenge identified by Tahir and Krogstie (n.d.).

To address this, this study aims to provide a way to guide the school population by developing the DLSU Campus Navigator System. With a desktop built program using Python as a programming language. This system provides a clean interface that allows users to select specific starting points and navigate to precise room-level destinations within the buildings of the DLSU campus. Using step-by-step instructions and the floor plans of each level, it displays the exact location of a room once they enter the target building, improving campus accessibility.

B. Problem Statement

In an academic institution like DLSU, students, faculty, and guests should be able to navigate through the university with ease using modern technology to assist them. Navigation systems put in place specifically for the university should be able to switch seamlessly between outdoor and indoor navigation and provide in depth information about the chosen destination. However, the navigation for DLSU relies solely on static and non-interactive maps that are located on each floor and a general campus map available online. This makes it difficult to find specific rooms, especially for those who are unfamiliar with the campus layout. Multi-level buildings like Andrew Gonzalez Hall are examples of complex structural designs that traditional 2D maps fail to represent accurately. Currently, there is no efficient way to traverse the campus premises. This results in wayfinding anxiety, academic tardiness, and inaccessibility for those with mobility issues by letting them figure out the campus layout without assistance. These issues highlight the need for a more efficient and interactive campus navigation system that is accessible for everyone and can provide accurate indoor and outdoor navigation by providing seamless room and inter-building routing within the DLSU campus.

C. Objectives

1. General Objectives

- The program aims to create a map that helps the user navigate the DLSU Manila Campus using official maps and floor plans.

2. Specific Objectives

- The program aims to guide users through the campus, helping them build familiarity with its layout.
- The program aims to help users navigate through the buildings of the campus.

D. Significance of the Project

Navigation within large and complex school campuses like DLSU can be a common problem for new students, visitors, and newcomers, this can be quite confusing. New students like us often had difficulties locating specific classrooms, offices, and rooms because of unfamiliarity. But with this problem, there is also a solution; a campus navigation system. This project provides an easy and practical solution to confusing layouts and lost students by giving clear directions.

For students, this project comes in handy, especially during the first few weeks of classes where no one knows the location of their rooms yet. By giving the clear directions to specific rooms, this reduces the time wasted searching for them and minimizes being late for class. For faculty and staff, this makes way for smoother campus operations by letting everyone know the exact location of the room/facility in question. Visitors and parents also benefit greatly from this whenever they have to look for an office or hub, most commonly for payments and documents.

Additionally, this project will serve as a stepping stone towards a better campus navigation in the future. This can be used as a base, a starting point for developing improved navigation for campus. As it demonstrates the basic

application of pathfinding algorithms and UI design, future projects for students and IT professionals of universities will be able to improve this project beyond its full potential.

II. Review of Related Literature

- **Indoor Navigation Systems**

Smartphones are used everywhere and keep getting improved. With that, individuals can use their phones to locate anything, anytime, anywhere. It has also been useful for individuals to know where they are and how to get to their desired destination. Through the Global Positioning System (GPS), it works well in guiding individuals where they are walking or driving, but often fails to work indoors. In addressing this problem, researchers have innovated a system in the mobile phone that could smoothly switch to show the maps of both indoor and outdoor spaces for the users to refrain from getting lost when moving between indoor or outdoor spaces. The system functions by applying different tools inside the phone and it uses tiny sensors that can detect movements like steps or turns. It also uses satellites from the Global Navigation Satellite System (GNSS), which has a similar function to GPS. The software of the system was constructed using a program for developers called Eclipse and the maps were made through a mapping service called the Baidu Map, which is similar to Google Maps. Indoor Maps were added aside from the Baidu Map for individuals to navigate rooms, hallways, and floors of an establishment. This system works accurately, where it displays the real-time location and even continuously functions in areas where the GPS signals are blocked or in other terms, the dead zones.

The study about the seamlessness of the position system on the mobile navigation map tackled on how the system addressed the main problems. The system automatically switches maps between indoors and outdoors depending on the signals from the satellite as the GPS does not function indoors. It is designed to make indoor navigation more accurate through applying smart tricks, such as

counting steps, fixing direction, especially when turning, and estimating how long each step takes. Aside from the system, a mobile app was also developed, where the indoor maps overlay the outdoor maps for their users to view everything clearly. The indoor facilities, such as the rooms, floors, and others, were measured accurately using the laser tool and were marked using number cards. The thickness of the wall was not included in the map to make it simple and the rooms were laid out as rectangles, creating a map that is very easy to understand. Apart from the laser tool, Differential GPS was also utilized to check the room dimensions and measure the edges of the building. With the right sensors, accurate maps, and the software, the system became a guide for users to find their way and that is very useful for schools, offices, and other establishments (Zhang et al, 2017).

- **Dijkstra Algorithm**

Dijkstra's Algorithm is a simple pathfinding algorithm that finds the shortest path from point A to point B. It starts with creating a table of nodes in a network and a separate column to which you will list nodes that have already been visited, the network should already know the distance between one node and its neighbor. After you've created the table, select the starting node and add it to the Visited list. After that, locate each immediate neighbor of the starting node that has not yet been visited, if there are more than one unvisited neighbor, you go to the one with the shortest distance. Calculate the cost to travel from the node to each of these neighbors, and enter these values into the table, repeat this process until you've visited every node. To get the shortest path to any node from start, you go to the neighbor with the shortest distance, following the path that was taken while mapping the network (Javaid, 2013).

- **Approaches to Design of Public Navigation Systems in Public Institutions**

Surina (2019) explored the user-centered and communicative aspects of navigation in public institutions. They emphasized that navigation systems extend beyond directional signage to create a comprehensive method of accompanying

people in an unfamiliar architectural environment. This study establishes that wayfinding in educational environments requires navigation systems that are visually intuitive, internationally understandable through symbols and pictograms, and integrates itself with the institution's identity. Additionally, the study emphasizes that university navigation systems must maintain coherence throughout multiple buildings and should all be understandable from the entrance up until the destination. This work ultimately highlights that navigation systems are an integration of all visual communication types that when optimized, can reduce visitor confusion and decrease reliance on personnel for directions.

- **Design and development of Google glass-based campus navigation system**

According to the study of Carag, J. et al. (2016), the incorporates wearable technology, such as Google Glass, into everyday life by developing a navigation system for the Manila Campus of De La Salle University. The project enables users who have the application and equipment to navigate both indoor and outdoor areas of the campus. Through the Google Glass main menu, users can select their desired destination from a list of locations. Once selected, the device displays a map with a routing path from the user's starting point to the chosen destination. For indoor navigation, a blue dot indicator shows the user's real-time position, making it easier to move through classrooms, offices, and other facilities within the campus.

The system also includes a data logging feature that records user movement using QR-coded checkpoints to manage transitions between indoor and outdoor navigation. Its Indoor Positioning System (IPS) utilizes magnetic positioning technology by capturing geomagnetic fingerprints of indoor spaces through the Indoor Atlas API. To determine the most efficient path, the application applies the spanning tree concept as its routing algorithm. Overall, this project serves as a proof of concept for future implementations in other environments such as malls, government buildings, and similar large-scale establishments.

- **Indoor/Outdoor Positioning and Navigating System Set-Up in a University Campus**

This study by Biczók et al (2014). discusses the implementation of MazeMap, a digital navigation system deployed at the Norwegian University of Science and Technology (NTNU). The system was designed to help students and visitors easily navigate a large and complex campus environment that includes both indoor and outdoor areas.

The researchers explain that traditional GPS systems work well outdoors but are unreliable inside buildings. To solve this, MazeMap integrates indoor maps with positioning technologies, allowing users to locate themselves and receive turn-by-turn directions across classrooms, offices, and campus pathways.

The study also analyzes user movement patterns and system usage data. Findings show that students frequently rely on digital navigation tools, especially during the start of semesters when new students are unfamiliar with campus layouts. The system improved navigation efficiency and reduced confusion in complex building structures.

Overall, the paper demonstrates that campus-specific navigation systems improve accessibility, user experience, and mobility within university environments. It supports the idea that a campus navigator project can provide practical value by helping users efficiently find locations within a school setting.

- **How Navigation Systems can help with a user's Spatial Knowledge**

According to a study by Brugger et al (2019), Navigation systems on mobile devices allow its users to efficiently find their way around a space. The study found that active participation and decision-making combined with the navigation system's path allowed its users to improve their spatial awareness and navigation efficiency.

The study found that while current navigation systems are efficient, it distracts the user's attention. They studied how people's behavior shifts in the presence of a navigation system like a GPS.

The study concluded that groups with users' decisions on where to mark landmarks and where to allocate attention (allocation of attention by human and self-localization by system) showed 82% success rates in finding the exact same route back.

- **Detecting large crowds and rerouting to a faster path**

Navigation systems of today are often implementing a crowd-awareness feature to their systems to improve travel times and user experience. A relevant study to this is the work of Gu et al. (2022), which focuses on predicting pedestrian flow and recommending routes based on the flow. With this study, we can conclude that crowd movement is not static, but is influenced by factors like location, events, and heat. Because of this, certain areas may be highly congested and crowded at times.

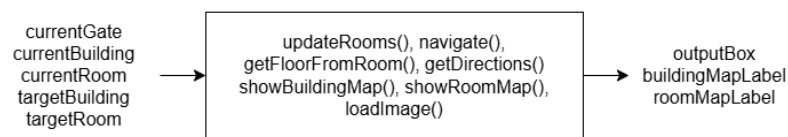
Gu et al. (2022) proposes a system that analyzes and predicts the flow of the crowd to recommend optimal routes. By detecting areas with high pedestrian traffic, the system can adjust routes, if possible, to the fastest one to avoid congestion. This helps greatly by removing the need for delays caused by crowded places. With this system, it's greatly emphasized that real-time or predictive data is very crucial for enhancing navigation systems, especially indoors where there are many people who move from place to place.

This concept can be directly applied to this proposed project. The system can analyze the number of users of the application within a certain area. When a big number of users are concentrated in a spot, the system can interpret this as a possible congestion point and redirect to a more efficient, less crowded route.

III. Methodology

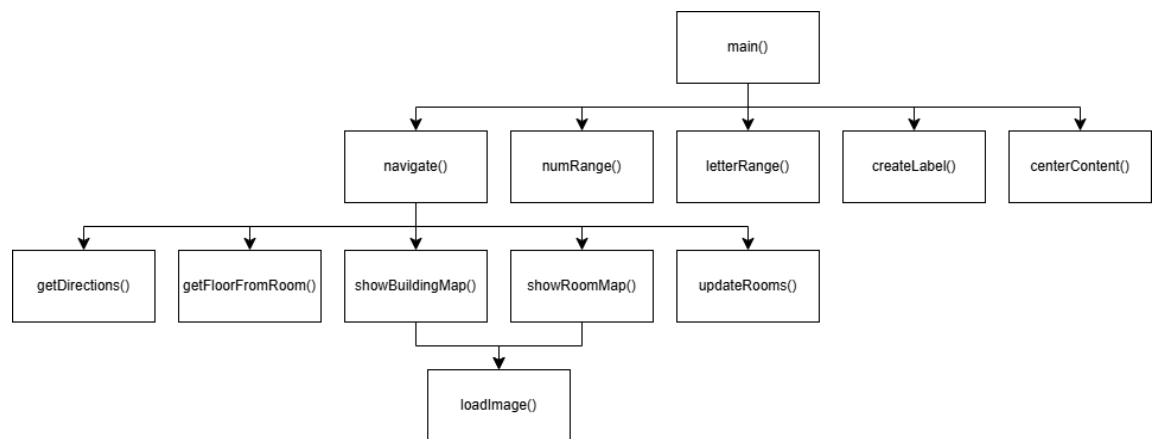
This section shows the systematic approach used to develop the DLSU Campus Navigator System, from initial design to implementation. The program applies the basic commands such as the conditional statements, switch/case structure, arrays, modules, and opening a file. Additionally, the methodology follows a structured programming process that uses user inputs into navigation outputs.

A. Conceptual Framework – IPO Chart/Diagram (Input-Process Output-Chart/Diagram)



IPO Chart

B. Hierarchy Chart



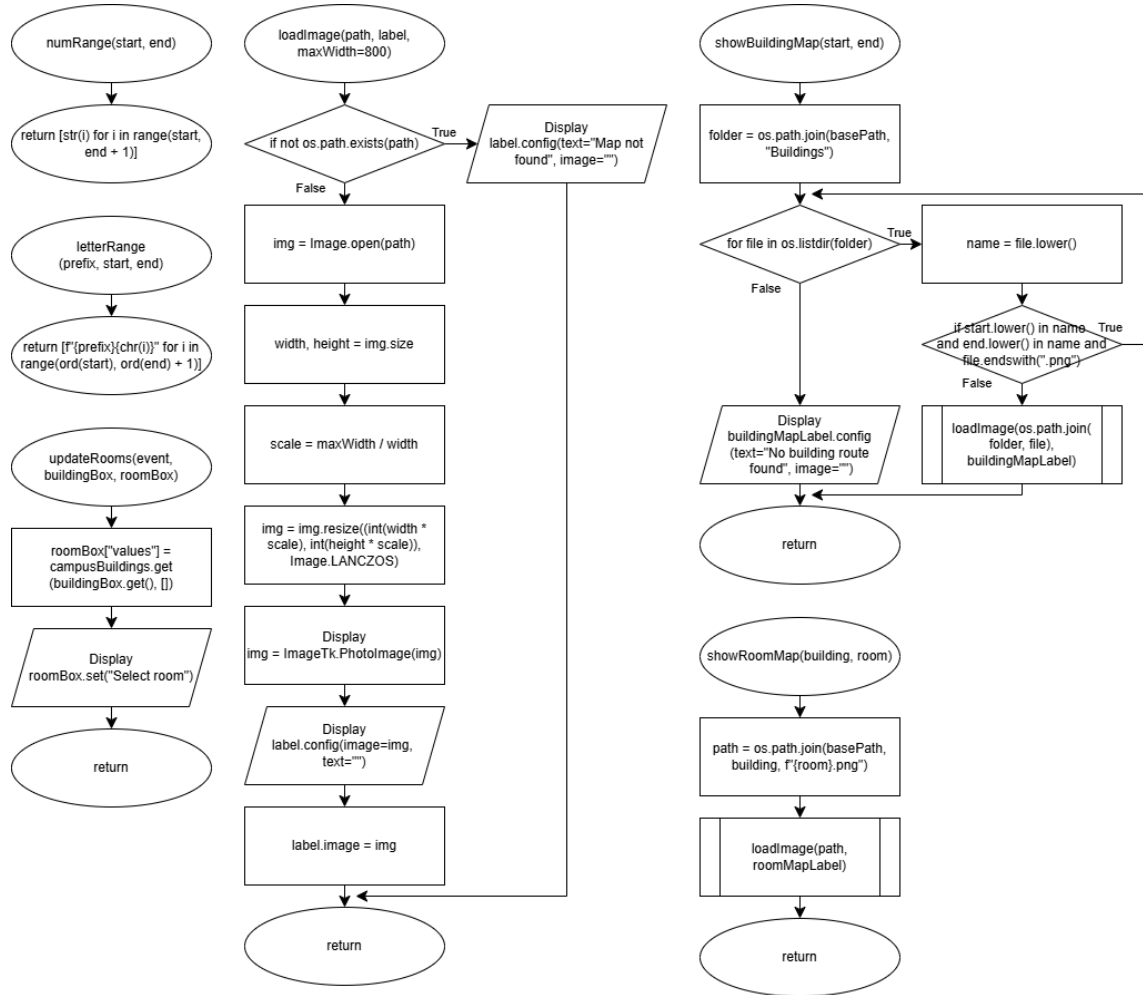
Hierarchy Chart

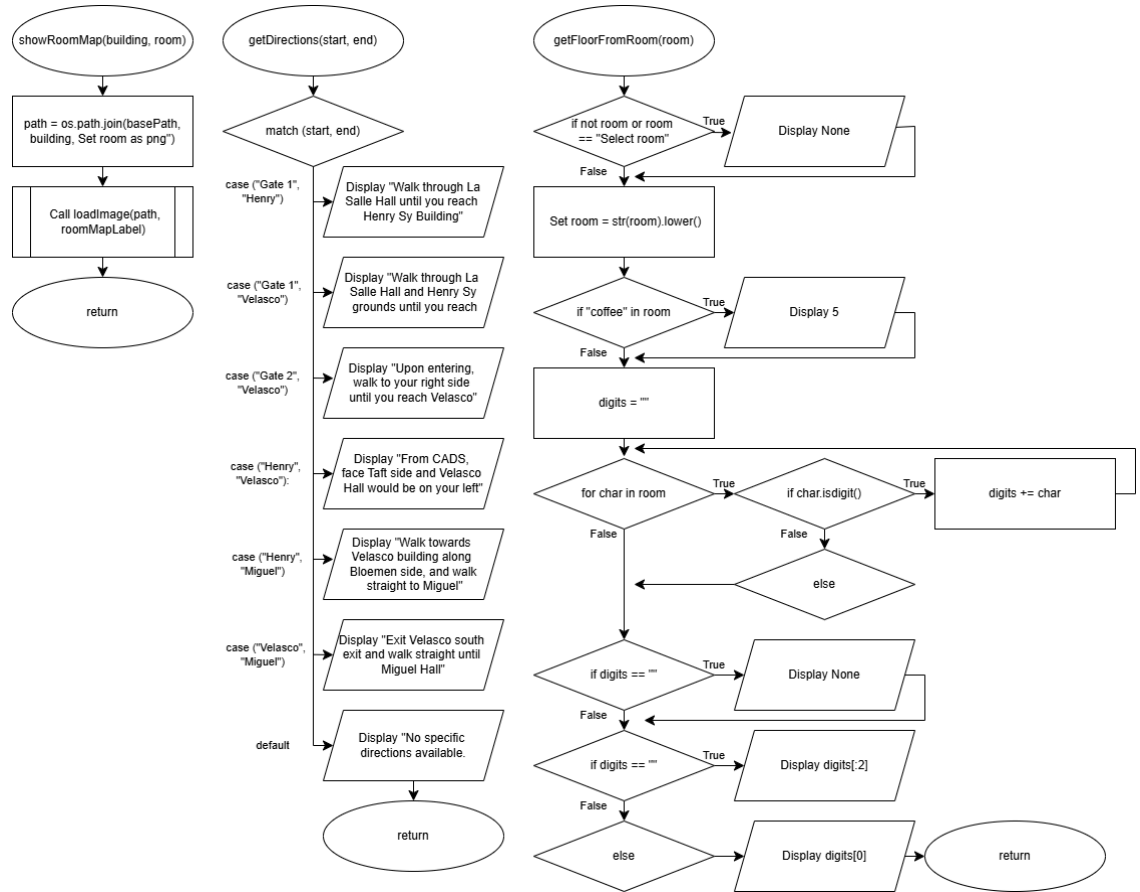
C. Algorithm

1. Input currentGate or currentBuilding and currentRoom
2. Input targetBuilding and targetRoom
3. List all buildings and rooms
4. Determine the starting point is from the gate, from another building, or within the same building
5. Determine the floor of the room by its first number
6. Determine the text directions

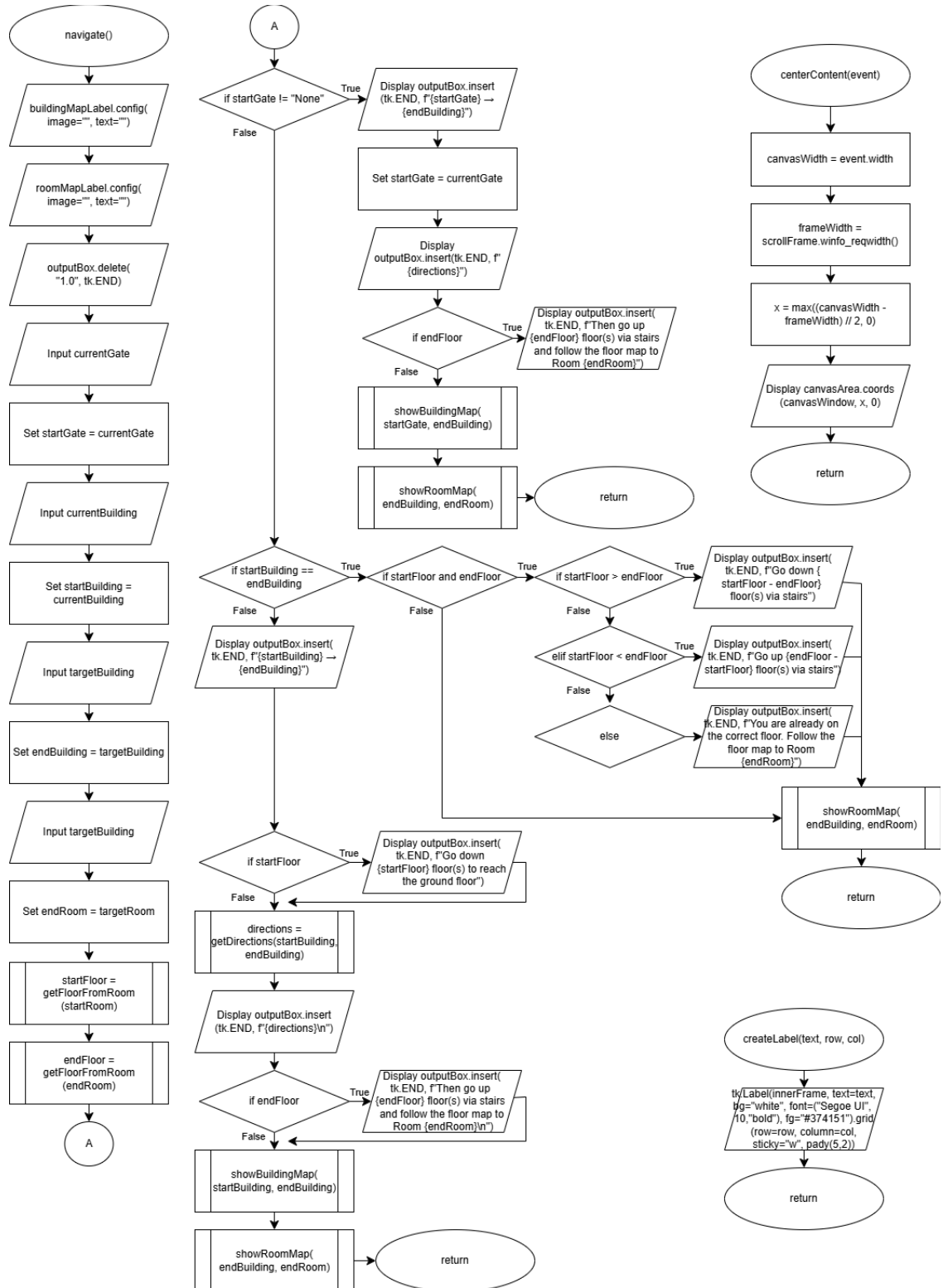
7. Display the text directions and the 2D layout map with directions between buildings and the floor layout of the target room

D. Flowchart

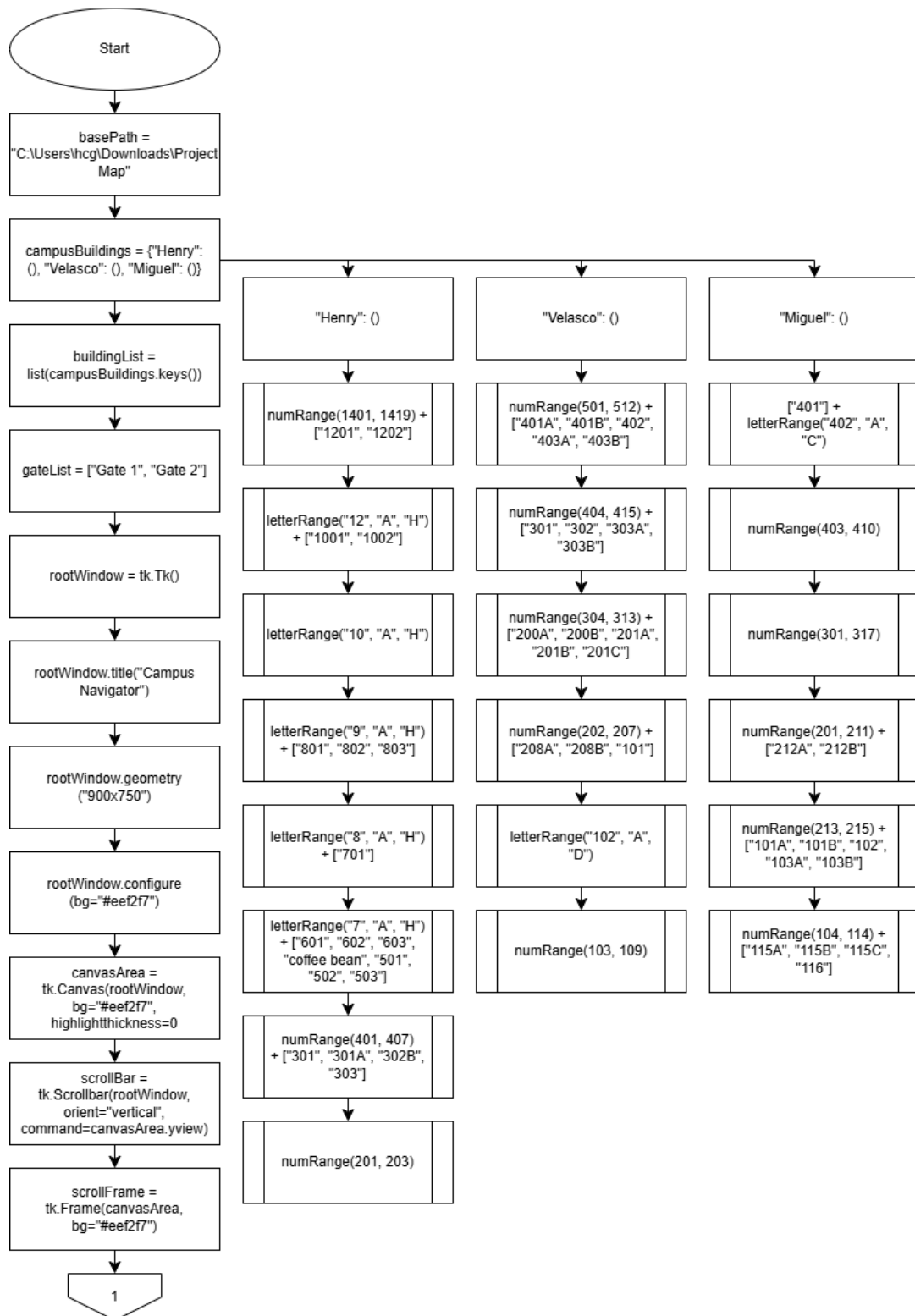




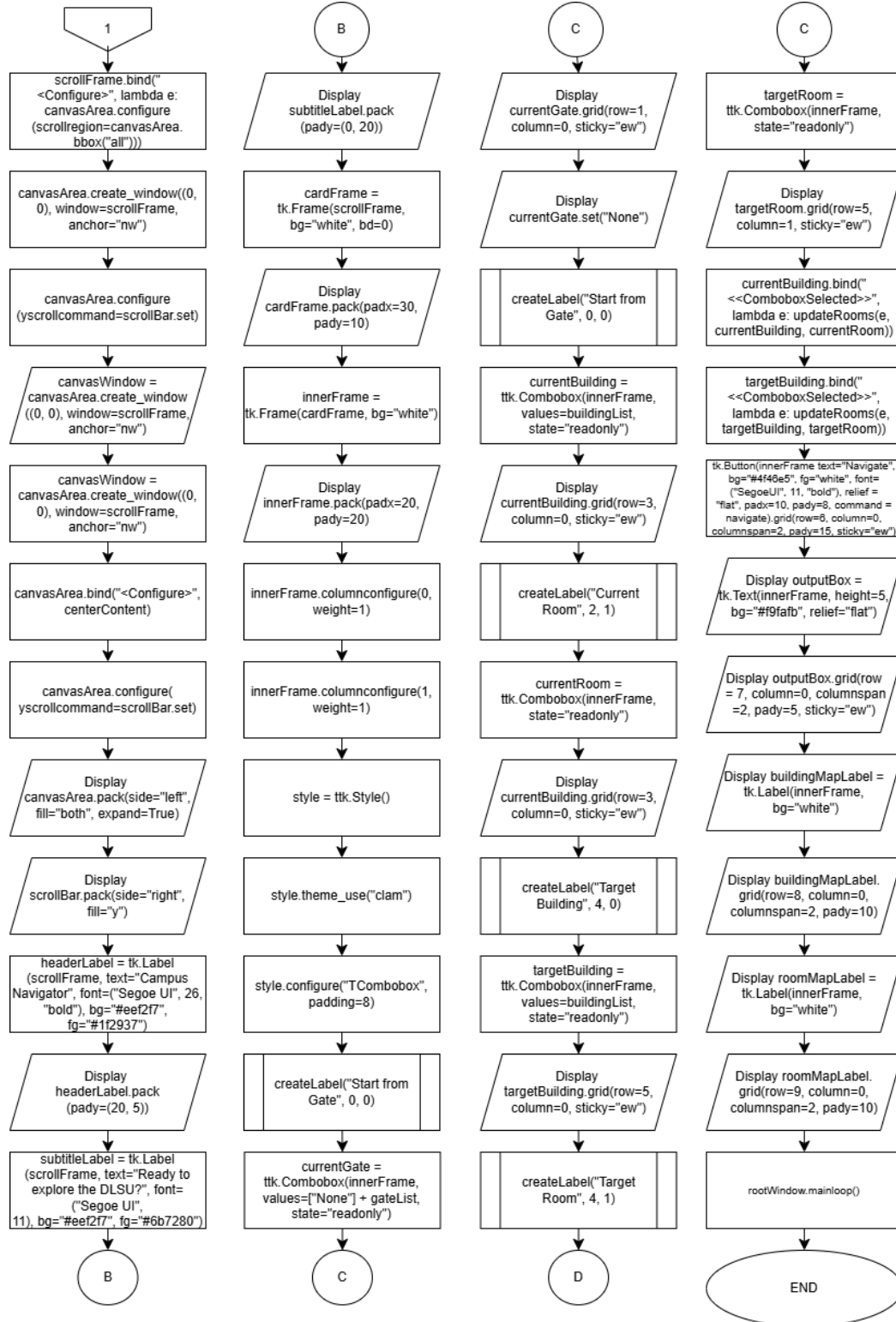
Flowchart Page 2



Flowchart Page 3



Flowchart Page 4



E. Pseudocode

1. import tkinter library as tk
2. Import ttk module from tkinter
3. Import Image and ImageTk from PIL library
4. import os module
- 5.
6. Declare String currentGate, currentBuilding, currentRoom, targetBuilding, targetRoom, startFloor, endFloor, directions
7. Declare Integer width, height, scale, digits
- 8.
9. Set basePath = "C:\\Users\\hcg\\Downloads\\Project Map"
- 10.
11. //List of Rooms,Buildings, and Gates
12. Module numRange(start, end)
13. return [str(i) for i in range(start, end + 1)]
14. End Module
- 15.
16. Module letterRange(prefix, start, end):
17. return [f"{prefix}{chr(i)}" for i in range(ord(start), ord(end) + 1)]
18. End Module
- 19.
20. Set campusBuildings = {
21. "Henry": (
22. Call numRange(1401, 1419) +
23. ["1201", "1202"] +
24. Call letterRange("12", "A", "H") +
25. ["1001", "1002"] +
26. Call letterRange("10", "A", "H") +
27. Call letterRange("9", "A", "H") +
28. ["801", "802", "803"] +
29. Call letterRange("8", "A", "H") +

```

30.     ["701"] +
31.     Call letterRange("7", "A", "H") +
32.     ["601", "602", "603", "coffee bean",
33.     "501", "502", "503"] +
34.     Call numRange(401, 407) +
35.     ["301", "301A", "302B", "303"] +
36.     Call numRange(201, 203)
37. ),
38.
39. "Velasco": (
40.     Call numRange(501, 512) +
41.     ["401A", "401B", "402", "403A", "403B"] +
42.     Call numRange(404, 415) +
43.     ["301", "302", "303A", "303B"] +
44.     Call numRange(304, 313) +
45.     ["200A", "200B", "201A", "201B", "201C"] +
46.     Call numRange(202, 207) +
47.     ["208A", "208B", "101"] +
48.     Call letterRange("102", "A", "D") +
49.     Call numRange(103, 109)
50. ),
51. "Miguel": (
52.     ["401"] +
53.     Call letterRange("402", "A", "C") +
54.     Call numRange(403, 410) +
55.     Call numRange(301, 317) +
56.     Call numRange(201, 211) +
57.     ["212A", "212B"] +
58.     Call numRange(213, 215) +
59.     ["101A", "101B", "102", "103A", "103B"] +
60.     Call numRange(104, 114) +

```

```

61.      ["115A", "115B", "115C", "116"]
62.  )
63. }
64.
65. Set buildingList = list(campusBuildings.keys())
66. Set gateList = ["Gate 1", "Gate 2"]
67.
68. //Adjusts Room Options Depending on the Building
69. Module updateRooms(event, buildingBox, roomBox)
70.     Set roomBox["values"] = Get campusBuildings( Get buildingBox, [])
71.     Display roomBox = "Select room"
72. End Module
73.
74. Module loadImage(path, label, maxWidth=800)
75.     If path not exist Then
76.         Display "Map not found", image=" "
77.
78.     Set img = open path
79.     Set width, height = img size
80.     Set scale = maxWidth / width
81.     Display img = resize (((width * scale), (height * scale)),
        Image.LANCZOS)
82.     Set img = Tk
83.
84.     Display label (Set image=img, Set text="")
85.     Set label.image = img
86. End Module
87.
88. //Displays Map with Route Lines
89. Module showBuildingMap(start, end)
90.     Set folder = os.path.join(BASE_PATH, "Buildings")

```

```

91.
92.   For file in os.listdir(folder)
93.       Set name = file.lower()
94.       If start.lower() in name and end.lower() in name and
           file.endswith(".png") Then
95.           Call loadImage(os.path.join(folder, file), buildingMapLabel)
96.       End If
97.   End For
98.
99.   Display text="No building route found", image=""
100.   End Module
101.
102.   //Displays Room Layout
103.   Module showRoomMap(building, room):
104.       Set path = os.path.join(basePath, building, Set room as png")
105.       Call loadImage(path, roomMapLabel)
106.   End Module
107.
108.   // Directions
109.   Module getDirections(start, end)
110.       Select(start, end):
111.           case ("Gate 1", "Henry"):
112.               Display "Walk through La Salle Hall until you reach Henry Sy
                   Building"
113.           case ("Gate 1", "Velasco"):
114.               Display "Walk through La Salle Hall and Henry Sy grounds
                   until you reach Velasco"
115.           case ("Gate 2", "Velasco"):
116.               Display "Upon entering, walk to your right side until you reach
                   Velasco"
117.           case ("Henry", "Velasco"):

```

```

118.         Display "From CADS, face Taft side and Velasco Hall would
           be on your left"
119.         case ("Henry", "Miguel"):
120.             Display "Walk towards Velasco building along Bloemen side,
           and walk straight to Miguel"
121.         case ("Velasco", "Miguel"):
122.             Display "Exit Velasco south exit and walk straight until Miguel
           Hall"
123.         case _:
124.             Display "No specific directions available."
125.     End Select
126. End Module
127.
128. //Function to Extract Floor Number from Room Name
129. Module getFloorFromRoom(room):
130.     If not room or room == "Select room" Then
131.         Display None
132.     End If
133.
134.     Set room = str(room).lower()
135.
136.     If "coffee" in room Then
137.         Display 5
138.     End If
139.
140.     Set digits = ""
141.     for char in room:
142.         If char.isdigit() Then
143.             Set digits += char
144.         Else Then
145.             break

```

```

146.         End If
147.
148.     If digits == "" Then
149.         return None
150.     End If
151.
152.     //FIXED - always correct floor extraction
153.     If len(digits) >= 4 Then
154.         Display digits[:2]
155.     Else If Then
156.         Display digits[0]
157.     End If
158.
159. //Choosing the Map to Display and the Output Text
160. Module navigate():
161.     Display buildingMapLabel.config(image="", text="")
162.     Display roomMapLabel.config(image="", text="")
163.     Display outputBox.delete("1.0", tk.END)
164.
165.     Input currentGate
166.     Set startGate = currentGate
167.     Input currentBuilding
168.     Set startBuilding = currentBuilding
169.     Input targetBuilding
170.     Set endBuilding = targetBuilding
171.     Input targetBuilding
172.     Set endRoom = targetRoom
173.
174.     Set startFloor = Call getFloorFromRoom(startRoom)
175.     Set endFloor = Call getFloorFromRoom(endRoom)
176.

```

```

177.      //From Gate
178.      If startGate != "None" Then
179.          Display outputBox.insert(tk.END, f" {startGate} →
              {endBuilding}\n\n")
180.
181.          Set directions = Call getDirections(startGate, endBuilding)
182.          Display outputBox.insert(tk.END, f" {directions}\n")
183.
184.          If endFloor Then
185.              Display outputBox.insert(tk.END,
186.                  f"Then go up {endFloor} floor(s) via stairs and follow the
                    floor map to Room {endRoom}\n")
187.          End If
188.
189.          Call showBuildingMap(startGate, endBuilding)
190.          Call showRoomMap(endBuilding, endRoom)
191.          return
192.      End If
193.
194.      //Same Building
195.      If startBuilding == endBuilding Then
196.          If startFloor and endFloor Then
197.              If startFloor > endFloor Then
198.                  Display outputBox.insert(tk.END,
199.                      f"Go down {startFloor - endFloor} floor(s) via stairs\n")
200.              Else If startFloor < endFloor Then
201.                  Display outputBox.insert(tk.END,
202.                      f"Go up {endFloor - startFloor} floor(s) via stairs\n")
203.              Else Then
204.                  Display outputBox.insert(tk.END,

```

```

205.          f"You are already on the correct floor. Follow the floor
           map to Room {endRoom}\n")
206.          End If
207.      End If
208.      Call showRoomMap(endBuilding, endRoom)
209.      return
210.  End If
211.
212.  //Building to Building
213.  Display outputBox.insert(tk.END, f"{startBuilding} →
           {endBuilding}\n\n")
214.
215.  //Step 1: Go Down First
216.  If startFloor Then
217.      Display outputBox.insert(tk.END,
218.          f"Go down {startFloor} floor(s) to reach the ground floor\n")
219.
220.  //Step 2: Walk Between Buildings
221.  Call directions = getDirections(startBuilding, endBuilding)
222.  Display outputBox.insert(tk.END, f"{directions}\n")
223.
224.  //Step 3: Go Up to Target Floor
225.  If endFloor Then
226.      Display outputBox.insert(tk.END,
227.          f"Then go up {endFloor} floor(s) via stairs and follow the floor
           map to Room {endRoom}\n")
228.
229.  Call showBuildingMap(startBuilding, endBuilding)
230.  Call showRoomMap(endBuilding, endRoom)
231.  End Module
232.

```

```

233. //GUI
234. Set rootWindow = Tk
235. Set title = Campus Navigator
236. Set size = 900x750
237. Set Color = #eef2f7
238.
239. //Scroll System
240. Set canvasArea = tk.Canvas(rootWindow, bg="#eef2f7",
    highlightthickness=0)
241. Set scrollBar = tk.Scrollbar(rootWindow, orient="vertical",
    command=canvasArea.yview)
242. Set scrollFrame = tk.Frame(canvasArea, bg="#eef2f7")
243.
244. Set scrollFrame.bind(
245.     "<Configure>",
246.     lambda e:
        canvasArea.configure(scrollregion=canvasArea.bbox("all"))
247. )
248.
249. Set canvasArea.create_window((0, 0), window=scrollFrame,
    anchor="nw")
250.
251. //Centering the Content in the Canvas
252. Module centerContent(event):
253.     Set canvasWidth = event.width
254.     Set frameWidth = scrollFrame.winfo_reqwidth()
255.
256.     Set x = max((canvasWidth - frameWidth) // 2, 0)
257.     Display canvasArea.coords(canvasWindow, x, 0)
258. End Module
259.

```

```
260. //Store Window Reference
261. Set canvasWindow = canvasArea.create_window((0, 0),
        window=scrollFrame, anchor="nw")
262.
263. //Bind Resize Event
264. Set canvasArea.bind("<Configure>", centerContent)
265. Set canvasArea.configure(yscrollcommand=scrollBar.set)
266.
267. Display canvasArea.pack(side="left", fill="both", expand=True)
268. Display scrollBar.pack(side="right", fill="y")
269.
270. //GUI Header
271. Set headerLabel = Campus Navigator
272. Display headerLabel
273.
274. Set subtitleLabel = Ready to explore the DLSU?
275. Display subtitleLabel
276.
277. //Card That Contains The Inputs and Outputs
278. Set cardFrame = tk.Frame(scrollFrame, bg="white", bd=0)
279. Display cardFrame.pack(padx=30, pady=10)
280.
281. Set innerFrame = tk.Frame(cardFrame, bg="white")
282. Display innerFrame.pack(padx=20, pady=20)
283.
284. Set innerFrame.columnconfigure(0, weight=1)
285. Set innerFrame.columnconfigure(1, weight=1)
286.
287. //Dropdown Style
288. Set style = ttk
289. Set style.theme_use("clam")
```

```

290. Set style.configure("TCombobox", padding=8)
291.
292. //Input System
293. Module createLabel(text, row, col):
294.     Display tk (innerFrame, text=text, bg="white",
295.         Set font=("Segoe UI", 10, "bold"),
296.         Set fg= "#374151"
297. End Module
298.
299. Call createLabel("Start from Gate", 0, 0)
300. Set currentGate = ttk.currentGate dropdown
301. Display currentGate.grid(row=1, column=0, sticky="ew")
302. Display currentGate.set("None")
303.
304. Call createLabel("Current Building", 2, 0)
305. Set currentBuilding = ttk.Combobox(innerFrame, values=buildingList,
    state="readonly")
306. Display currentBuilding.grid(row=3, column=0, sticky="ew")
307.
308. Call createLabel("Current Room", 2, 1)
309. Set currentBuilding = ttk.Combobox(innerFrame, ...)
310. Display currentBuilding.grid(row=3, column=0, sticky="ew")
311.
312. Call createLabel("Target Building", 4, 0)
313. Set targetBuilding = ttk.Combobox(innerFrame, ...)
314. Display targetBuilding.grid(row=5, column=0, sticky="ew")
315.
316. Call createLabel("Target Room", 4, 1)
317. Set targetRoom = ttk.Combobox(innerFrame, ...)
318. Display targetRoom.grid(row=5, column=1, sticky="ew")
319.

```

```

320. //Update Room Options Based on Building Selected
321. Set currentBuilding.bind("<<ComboboxSelected>>",
322.     lambda e: updateRooms(e, currentBuilding, currentRoom))
323. Set targetBuilding.bind("<<ComboboxSelected>>",
324.     lambda e: updateRooms(e, targetBuilding, targetRoom))
325.
326. //Button to Start Navigation
327. Set tk Button(innerFrame,
328.     Set text="Navigate",
329.     Set bg="#4f46e5",
330.     Set fg="white",
331.     Set font=("Segoe UI", 11, "bold"),
332.     Set relief="flat",
333.
334. //Output (Instructions)
335. Display outputBox = tk.Text(innerFrame, height=5, bg="#f9fafb",
    relief="flat")
336. Display outputBox.grid(row=7, column=0, columnspan=2, pady=5,
    sticky="ew")
337.
338. //Output (Maps)
339. Display buildingMapLabel = tk.Label(innerFrame, bg="white")
340. Display buildingMapLabel.grid(row=8, column=0, columnspan=2,
    pady=10)
341.
342. Display roomMapLabel = tk.Label(innerFrame, bg="white")
343. Display roomMapLabel.grid(row=9, column=0, columnspan=2,
    pady=10)
344.
345. //Keep the Window Running
346. Set rootWindow.mainloop()

```

IV. Results

- Result Set 1

Campus Navigator

Ready to explore the DLSU?

Start from Gate

Gate 1

Current Building

Current Room

Target Building

Henry

Target Room

1408

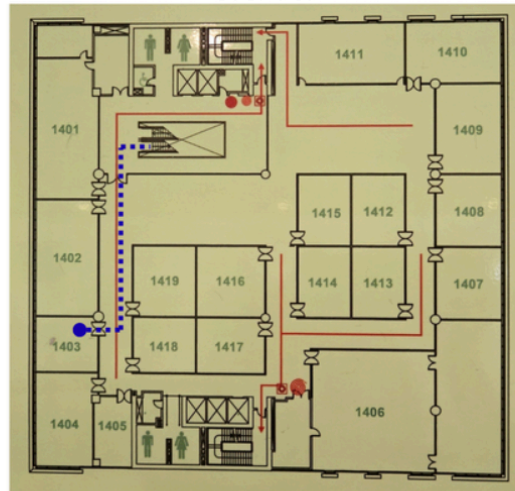
Navigate

Gate 1 → Henry

Walk through La Salle Hall until you reach Henry Sy Building
Then go up 14 floor(s) via stairs and follow the floor map to Room 1403



14TH FLOOR
HENRY SY



- **Result Set 2**

Campus Navigator

Ready to explore the DLSU?

Start from Gate

None

Current Building

Henry

Current Room

1403

Target Building

Miguel

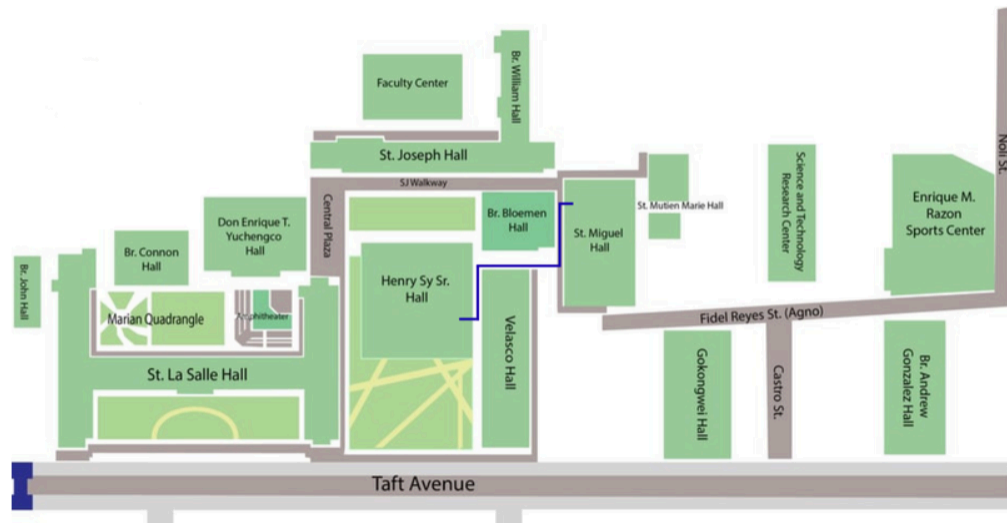
Target Room

206

Navigate

Henry → Miguel

Go down 14 floor(s) to reach the ground floor
Walk towards Velasco building along Bloemen side, and walk straight to Miguel
Then go up 2 floor(s) via stairs and follow the floor map to Room 206



2ND FLOOR ST. MIGUEL



- **Result Set 3**

Campus Navigator

Ready to explore the DLSU?

Start from Gate

None

Current Building

Velasco

Current Room

509

Target Building

Velasco

Target Room

306

Navigate

Go down 2 floor(s) via stairs



V. Discussion of Results

The results in sets 1-3 provide clear visual evidence of the DLSU Campus Navigator System's functionality even in different navigation scenarios such as gate to building, building to building, and room to room. Moreover, the program uses DLSU's official 2D floor plan paired with text instructions.

With the consistent results, the program achieves its stated objectives of providing map guided navigation and familiarization for DLSU Manila campus layout. Additionally, the program's hardcoded directions ensure reliability without GPS dependency, utilizing PNG based maps similar to MazeMap implementations.

However, the program's limitations include the system's use of fixed routing logic rather than using shortest path computation by Dijkstra's algorithm. Additionally, the functionality depends on locally stored PNG map files in the 'C:\Map' directory, which will display errors if images are missing or paths change.

VI. Analysis, Conclusion and Future Directives

The test results show the program is reliable even in different navigation scenarios such as entry gate to buildings, building to building, and room to room scenarios. Additionally, Its ability to analyze floor numbers from room numbers and update the room options based on the building selection shows reliable design implementation. Moreover, this analysis focuses on the performance consistency with no crashes occurred despite varying input combinations and the advantage of visual map integration, which reinforces textual directions for better user comprehension

In conclusion, this project successfully achieves its main objectives by providing 2D maps using DLSU's official floor plans. The program also enhances student's familiarity with DLSU Manila campus layout, and makes it easier to navigate between buildings. In addition, the use of fixed directions that match real campus pathways ensures

consistent performance and reliability, proving simple fixed direction navigational tools are valuable for schools and universities.

Future directives

- Use of algorithms in calculating the most efficient walking routes
- Addition of other buildings such as Andrew, Gokongwei, and Razon.

References

- Bacalla, I., Barcelona, A. L., Gisultura, F., Pacatang, R., Romarate, K., Cortes, V., & Cabanilla, A. (n.d.). Evaluating the Impact of Braille Signage on Wayfinding System: A Phenomenological Study of Campus Accessibility for Students with Visual Impairments. CNU Journal Higher Education. <https://jhe.researchcommons.org/journal/vol18/iss1/4/>
- Biczók, G., Martinez, S. D., Jelle, T., & Krogstie, J. (2014). Navigating MazeMap: Indoor/outdoor positioning and navigation system deployed at a university campus. arXiv. <https://arxiv.org/abs/1401.5297>
- Brügger, A., Richter, K., & Fabrikant, S. I. (2019). How does navigation system behavior influence human behavior? *Cognitive Research Principles and Implications*, 4(1), 5. <https://doi.org/10.1186/s41235-019-0156-5>
- Gu, J., Song, C., Ren, Z., Lu, L., Jiang, W., & Liu, M. (2022). Pedestrian Flow Prediction and Route Recommendation with Business Events. *Sensors*, 22(19), 7478. <https://doi.org/10.3390/s22197478>
- James, K., Carag, S. L., Emanuel, D., & Osbert, J. (2016). Design and development of Google glass-based campus navigation system. Animo Repository. https://animorepository.dlsu.edu.ph/etd_bachelors/7462/
- Javaid, A. (2013). Understanding Dijkstra Algorithm. SSRN Electronic Journal. 10.2139/ssrn.2340905.
- Surina, L. (2019). Approaches to design of navigation systems in public institutions. *IOP Conference Series Materials Science and Engineering*, 687(5), 055011. <https://doi.org/10.1088/1757-899x/687/5/055011>
- Tahir, R. & Krogstie, J. (n.d.). Impact of Navigation Aid and Spatial Ability Skills on Wayfinding

Performance and Workload in Indoor-Outdoor Campus Navigation: Challenges and Design.
Applied Sciences.

[https://nva.sikt.no/registration/0198cc61feb3-99163e8b-ec5c-47fa-8de1-c80245d4cf4e\](https://nva.sikt.no/registration/0198cc61feb3-99163e8b-ec5c-47fa-8de1-c80245d4cf4e)

Zhang et al. (2017, January 9). Implementation of smartphone seamless positioning system based on mobile navigation electronic map. IEEE XPLORE.

<https://ieeexplore-ieee-org.dlsu.idm.oclc.org/document/7809955>

Appendices

A. User's Manual

- If the starting point is from either Gate 1 or Gate 2 use the dropdown button from the first row labeled as “Start from Gate” then click one of the options.
- If the starting point is from a building use the dropdown button from the second row labeled as “Current Building” then click one of the options depending on the building the user is at. Then, use the dropdown button beside “Current Building” labeled as the “Current Room” depending on the specific location of the user inside the building.
- After selecting the current location, the user shall use the dropdown button from the third row labeled as “Target Building” to select the desired building and the dropdown button at the right labeled as “Target Room” to get the specific location inside the building.
- Press the Navigate button to display the directions and the map layout with routes.

B. Source Code

```
import tkinter as tk
from tkinter import ttk
from PIL import Image, ImageTk
import os

basePath = r"C:\Users\hcg\Downloads\Project Map"

#List of Rooms,Buildings, and Gates
def numRange(start, end):
    return [str(i) for i in range(start, end + 1)]

def letterRange(prefix, start, end):
    return [f'{prefix}{chr(i)}' for i in range(ord(start), ord(end) + 1)]

campusBuildings = {
```

```
"Henry": (  
  numRange(1401, 1419) +  
  ["1201", "1202"] +  
  letterRange("12", "A", "H") +  
  ["1001", "1002"] +  
  letterRange("10", "A", "H") +  
  letterRange("9", "A", "H") +  
  ["801", "802", "803"] +  
  letterRange("8", "A", "H") +  
  ["701"] +  
  letterRange("7", "A", "H") +  
  ["601", "602", "603", "coffee bean",  
    "501", "502", "503"] +  
  numRange(401, 407) +  
  ["301", "301A", "302B", "303"] +  
  numRange(201, 203)  
)
```

```
"Velasco": (  
  numRange(501, 512) +  
  ["401A", "401B", "402", "403A", "403B"] +  
  numRange(404, 415) +  
  ["301", "302", "303A", "303B"] +  
  numRange(304, 313) +  
  ["200A", "200B", "201A", "201B", "201C"] +  
  numRange(202, 207) +  
  ["208A", "208B", "101"] +  
  letterRange("102", "A", "D") +  
  numRange(103, 109)  
)
```

```
"Miguel": (  
  ["401"] +
```

```

letterRange("402", "A", "C") +
numRange(403, 410) +
numRange(301, 317) +
numRange(201, 211) +
["212A", "212B"] +
numRange(213, 215) +
["101A", "101B", "102", "103A", "103B"] +
numRange(104, 114) +
["115A", "115B", "115C", "116"]
)
}

```

```

buildingList = list(campusBuildings.keys())
gateList = ["Gate 1", "Gate 2"]

```

#Adjusts Room Options Depending on the Building

```

def updateRooms(event, buildingBox, roomBox):
    roomBox["values"] = campusBuildings.get(buildingBox.get(), [])
    roomBox.set("Select room")

```

#Displays Map Layout and Resizing the Image

```

def loadImage(path, label, maxWidth=800):
    if not os.path.exists(path):
        label.config(text="Map not found", image="")
        return

    img = Image.open(path)
    width, height = img.size
    scale = maxWidth / width
    img = img.resize((int(width * scale), int(height * scale)), Image.LANCZOS)
    img = ImageTk.PhotoImage(img)

```

```

label.config(image=img, text="")
label.image = img

#Displays Map with Route Lines
def showBuildingMap(start, end):
    folder = os.path.join(basePath, "Buildings")

    for file in os.listdir(folder):
        name = file.lower()
        if start.lower() in name and end.lower() in name and file.endswith(".png"):
            loadImage(os.path.join(folder, file), buildingMapLabel)
    return

    buildingMapLabel.config(text="No building route found", image="")

#Displays Room Layout
def showRoomMap(building, room):
    path = os.path.join(basePath, building, f"{room}.png")
    loadImage(path, roomMapLabel)

#Directions
def getDirections(start, end):
    match (start, end):
        case ("Gate 1", "Henry"):
            return "Walk through La Salle Hall until you reach Henry Sy Building"
        case ("Gate 1", "Velasco"):
            return "Walk through La Salle Hall and Henry Sy grounds until you reach Velasco"
        case ("Gate 2", "Velasco"):
            return "Upon entering, walk to your right side until you reach Velasco"

```

```

case ("Henry", "Velasco"):
    return "From CADs, face Taft side and Velasco Hall would be on your left"
case ("Henry", "Miguel"):
    return "Walk towards Velasco building along Bloemen side, and walk straight to
Miguel"
case ("Velasco", "Miguel"):
    return "Exit Velasco south exit and walk straight until Miguel Hall"
case _:
    return "No specific directions available."

```

#Function to Extract Floor Number from Room Name

```

def getFloorFromRoom(room):
    if not room or room == "Select room":
        return None

```

```

    room = str(room).lower()

```

```

    if "coffee" in room:
        return 5

```

```

    digits = ""
    for char in room:
        if char.isdigit():
            digits += char
        else:
            break

```

```

    if digits == "":
        return None

```

FIXED: always correct floor extraction

```
if len(digits) >= 4:
    return int(digits[:2])
else:
    return int(digits[0])
```

#Choosing the Map to Display and the Output Text

```
def navigate():
    buildingMapLabel.config(image="", text="")
    roomMapLabel.config(image="", text="")
    outputBox.delete("1.0", tk.END)
```

```
startGate = currentGate.get()
startBuilding = currentBuilding.get()
startRoom = currentRoom.get()
endBuilding = targetBuilding.get()
endRoom = targetRoom.get()
```

```
startFloor = getFloorFromRoom(startRoom)
endFloor = getFloorFromRoom(endRoom)
```

From Gate

```
if startGate != "None":
    outputBox.insert(tk.END, f"{startGate} → {endBuilding}\n\n")
```

```
directions = getDirections(startGate, endBuilding)
outputBox.insert(tk.END, f"{directions}\n")
```

```
if endFloor:
```

```
        outputBox.insert(tk.END,  
            f"Then go up {endFloor} floor(s) via stairs and follow the floor map to Room  
{endRoom}\n")
```

```
    showBuildingMap(startGate, endBuilding)  
    showRoomMap(endBuilding, endRoom)  
    return
```

```
#Same Building  
if startBuilding == endBuilding:  
    if startFloor and endFloor:  
        if startFloor > endFloor:  
            outputBox.insert(tk.END,  
                f"Go down {startFloor - endFloor} floor(s) via stairs\n")  
        elif startFloor < endFloor:  
            outputBox.insert(tk.END,  
                f"Go up {endFloor - startFloor} floor(s) via stairs\n")  
        else:  
            outputBox.insert(tk.END,  
                f"You are already on the correct floor. Follow the floor map to Room  
{endRoom}\n")
```

```
    showRoomMap(endBuilding, endRoom)  
    return
```

```
#Building to Building  
outputBox.insert(tk.END, f"{startBuilding} → {endBuilding}\n\n")
```

#Step 1: Go Down First

if startFloor:

outputBox.insert(tk.END,

f'Go down {startFloor} floor(s) to reach the ground floor\n")

#Step 2: Walk Between Buildings

directions = getDirections(startBuilding, endBuilding)

outputBox.insert(tk.END, f'{directions}\n")

#Step 3: Go Up to Target Floor

if endFloor:

outputBox.insert(tk.END,

f'Then go up {endFloor} floor(s) via stairs and follow the floor map to Room
{endRoom}\n")

showBuildingMap(startBuilding, endBuilding)

showRoomMap(endBuilding, endRoom)

#GUI

rootWindow = tk.Tk()

rootWindow.title("Campus Navigator")

rootWindow.geometry("900x750")

rootWindow.configure(bg="#eef2f7")

#Scroll System

canvasArea = tk.Canvas(rootWindow, bg="#eef2f7", highlightthickness=0)

scrollBar = tk.Scrollbar(rootWindow, orient="vertical", command=canvasArea.yview)

```
scrollFrame = tk.Frame(canvasArea, bg="#eef2f7")
```

```
scrollFrame.bind("<Configure>", lambda e:
```

```
canvasArea.configure(scrollregion=canvasArea.bbox("all")))
```

```
canvasArea.create_window((0, 0), window=scrollFrame, anchor="nw")
```

```
#Centering the Content in the Canvas
```

```
def centerContent(event):
```

```
    canvasWidth = event.width
```

```
    frameWidth = scrollFrame.winfo_reqwidth()
```

```
    x = max((canvasWidth - frameWidth) // 2, 0)
```

```
    canvasArea.coords(canvasWindow, x, 0)
```

```
#Store Window Reference
```

```
canvasWindow = canvasArea.create_window((0, 0), window=scrollFrame, anchor="nw")
```

```
#Bind Resize Event
```

```
canvasArea.bind("<Configure>", centerContent)
```

```
canvasArea.configure(yscrollcommand=scrollBar.set)
```

```
canvasArea.pack(side="left", fill="both", expand=True)
```

```
scrollBar.pack(side="right", fill="y")
```

```
#GUI Header
```

```
headerLabel = tk.Label(
```

```
    scrollFrame,
```

```
    text="Campus Navigator",
```

```
    font=("Segoe UI", 26, "bold"),
```

```
    bg="#eef2f7",
```

```

        fg="#1f2937"
    )
    headerLabel.pack(pady=(20, 5))

    subtitleLabel = tk.Label(
        scrollFrame,
        text="Ready to explore the DLSU?",
        font=("Segoe UI", 11),
        bg="#eef2f7",
        fg="#6b7280"
    )
    subtitleLabel.pack(pady=(0, 20))

    #Card That Contains The Inputs and Outputs
    cardFrame = tk.Frame(scrollFrame, bg="white", bd=0)
    cardFrame.pack(padx=30, pady=10)

    innerFrame = tk.Frame(cardFrame, bg="white")
    innerFrame.pack(padx=20, pady=20)

    innerFrame.columnconfigure(0, weight=1)
    innerFrame.columnconfigure(1, weight=1)

    #Dropdown Style
    style = ttk.Style()
    style.theme_use("clam")
    style.configure("TCombobox", padding=8)

    #Input System
    def createLabel(text, row, col):
        tk.Label(innerFrame, text=text, bg="white",

```

```
font=("Segoe UI", 10, "bold"),  
fg="#374151").grid(row=row, column=col, sticky="w", pady=(5,2))
```

```
createLabel(" 🏠 Start from Gate", 0, 0)  
currentGate = ttk.Combobox(innerFrame, values=["None"] + gateList, state="readonly")  
currentGate.grid(row=1, column=0, sticky="ew")  
currentGate.set("None")
```

```
createLabel(" 📍 Current Building", 2, 0)  
currentBuilding = ttk.Combobox(innerFrame, values=buildingList, state="readonly")  
currentBuilding.grid(row=3, column=0, sticky="ew")
```

```
createLabel("Current Room", 2, 1)  
currentRoom = ttk.Combobox(innerFrame, state="readonly")  
currentRoom.grid(row=3, column=1, sticky="ew")
```

```
createLabel(" 📍 Target Building", 4, 0)  
targetBuilding = ttk.Combobox(innerFrame, values=buildingList, state="readonly")  
targetBuilding.grid(row=5, column=0, sticky="ew")
```

```
createLabel("Target Room", 4, 1)  
targetRoom = ttk.Combobox(innerFrame, state="readonly")  
targetRoom.grid(row=5, column=1, sticky="ew")
```

```
#Update Room Options Based on Building Selected  
currentBuilding.bind("<<ComboboxSelected>>",  
    lambda e: updateRooms(e, currentBuilding, currentRoom))  
targetBuilding.bind("<<ComboboxSelected>>",  
    lambda e: updateRooms(e, targetBuilding, targetRoom))
```

```
#Button to Start Navigation
```

```
tk.Button(innerFrame,
          text="Navigate",
          bg="#4f46e5",
          fg="white",
          font=("Segoe UI", 11, "bold"),
          relief="flat",
          padx=10,
          pady=8,
          command=navigate).grid(row=6, column=0, columnspan=2, pady=15,
                                sticky="ew")
```

#Output (Instructions)

```
outputBox = tk.Text(innerFrame, height=5, bg="#f9fafb", relief="flat")
outputBox.grid(row=7, column=0, columnspan=2, pady=5, sticky="ew")
```

#Output (Maps)

```
buildingMapLabel = tk.Label(innerFrame, bg="white")
buildingMapLabel.grid(row=8, column=0, columnspan=2, pady=10)
```

```
roomMapLabel = tk.Label(innerFrame, bg="white")
roomMapLabel.grid(row=9, column=0, columnspan=2, pady=10)
```

#Keep the Window Running

```
rootWindow.mainloop()
```